# Introduction à PyQt

Par Harsh - Traducteur : Thibaut Cuvelier

Date de publication : 31 mai 2011

Dernière mise à jour : 24 juin 2011

Cet article est une introduction au framework de développement d'applications graphiques PyQt avec Python et le framework Qt de Nokia.

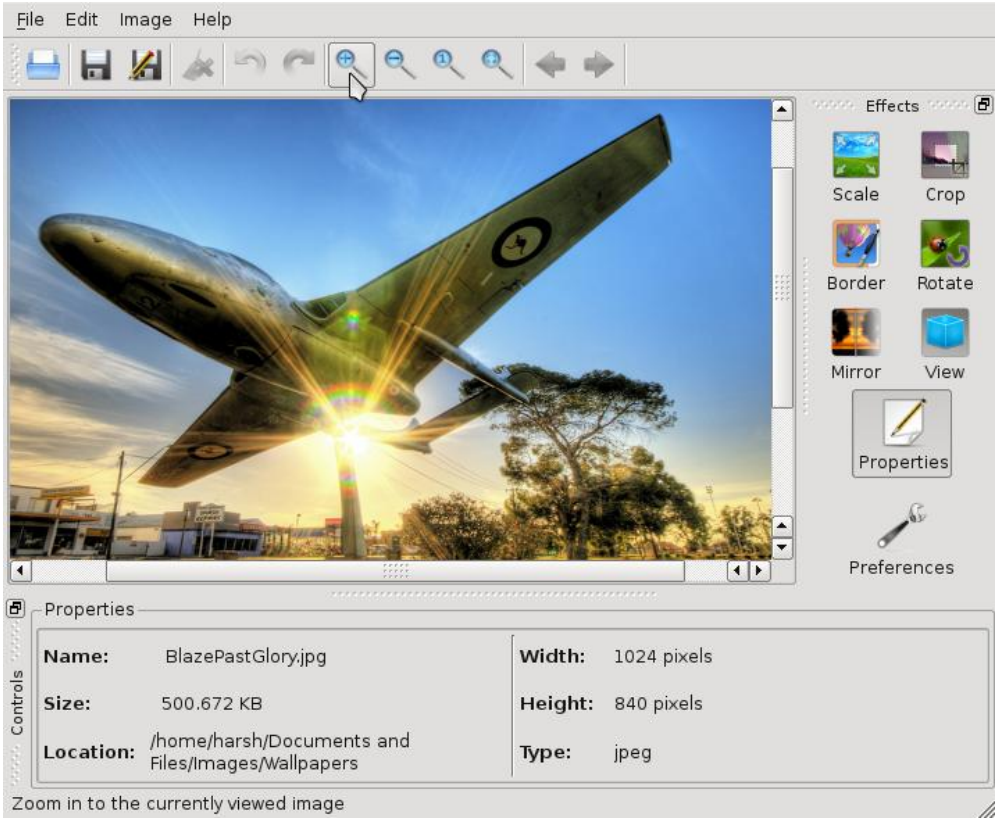
I - Prétexte....3
II - Prérequis....4
III - Code d'exemple....4
IV - Explications....5
V - Notes....5
VI - Autre programme de test....5
VII - Remerciements....8


Copyright ® 2009 Harsh. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts. Cette page est déposée à la SACD.

# I - Prétexte

Vous êtes-vous déjà demandé comme écrire facilement de bons programmes avec Python ? Des programmes qui pourraient se lancer sur toute plateforme - de Windows à Mac OS X en passant par les diverses variantes de Linux ? De beaux programmes, d'apparence native, qui fonctionnent bien ?

Voici une solution : PyQt. Avec lui, on peut créer des applications complètes et fonctionnelles qui pourraient ressembler à ceci :




Qt est un framework pour applications cross-plateformes, libre (sous la LGPL) de Nokia, accessible en C++ et en Java, les GUI étant sa fonctionnalité phare.

PyQt n'est qu'un binding Python des bibliothèques C++ fournies par Qt. En utilisant PYQt, on peut écrire des applications au look net qui font beaucoup de choses... et les écrire facilement.



Une série d'articles, commençant par celui-ci, vont tenter d'apprendre aux programmeurs intéressés par l'utilisation de cette merveille à développer des applications avec elle.


Copyright ® 2009 Harsh. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts. Cette page est déposée à la SACD.
http://tcuvelier.developpez.com/tutoriels/pyqt/introduction-pyqt/

## II - Prérequis

Les prérequis minimaux sont la connaissance basique de la programmation avec Python et des connaissances générales sur les GUI et les concepts de programmation orientée événement.

Deux sources primaires d'information : **le site de Qt, par Nokia** et **celui de PyQt, par Riverbank Computing**.

Télécharger et installer PyQt sur Linux est facile sous Linux, grâce aux divers gestionnaires de paquets. Regardez les liens ci-dessus et les parties réservées au téléchargement sur ces sites pour l'installation sur d'autres plateformes. Une aide à l'installation est aussi fournie par Riverbank Computing pour l'installation de PyQt4 sur n'importe quel OS.

## III - Code d'exemple

Commençons par la même chose qu'à peu près tout programmeur : le fameux *Hello World*.

Voici une version fortement documentée :

```
import sys

# Importation des classes de Qt nécessaires.

from PyQt4.QtGui import QLabel, QApplication

# On utilise la syntaxe from x import *, parce que
# tous les objets de Qt commencent par un Q et l'on
# n'aura pas de problème d'espace de noms ainsi.

if __name__=='__main__':

    App = QApplication(sys.argv)

    # Tous les programmes Qt doivent posséder une
    # instance de QApplication.

    # On passe sys.argv comme argument, car Qt est
    # expert en la gestion de certaines options
    # par défaut en ligne de commande, comme le style,
    # la taille, etc.

    Label = QLabel( "Hello World!" )

  # QLabel est la classe fournissant un simple label

    Label.show()

    # Comme dans la majorité des boîtes à outils pour GUI,
    # on doit préciser manuellement au widget qu'il doit
    # s'afficher.

    App.exec_()

    # Remarquez l'underscore après exec pour éviter la confusion
    # avec la fonction exec() standard de Python

    # exec_() démarre la boucle principale de l'application,
    # comme la fonction main() d'autres outils.
```

Voici ce à quoi ça ressemble une fois lancé, une simple application qui affiche le texte si populaire :





Copyright ® 2009 Harsh. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts. Cette page est déposée à la SACD.
http://tcuvelier.developpez.com/tutoriels/pyqt/introduction-pyqt/

## IV - Explications

On commence par initialiser, sous __main__, une instance de QApplication, app. Chaque application GUI doit avoir cette instance. Elle gère le fonctionnement de l'application et en est le thread principal. Sans elle, Qt va afficher une erreur et ne lancera aucune GUI.

Ensuite, on crée un label, un widget simple affichant du texte, utilisé dans de nombreux formulaires pour décrire les champs d'entrée et d'autres choses. Qt l'appelle QLabel, il réside sous l'espace de noms QtGui. On en crée un avec le simple texte *Hello World* sous le nom de Label.

Ensuite, on appelle la méthode show() du label, pour qu'il apparaisse à l'écran. Cette méthode est applicable à tous les enfants de QWidget, dont QLabel. Appeler cette méthode dessine la fenêtre du widget à l'écran.

Finalement, on démarre la boucle de l'application, celle responsable du fonctionnement de l'application comme une boucle infinie attendant une interaction avec l'utilisateur. Cela se fait par la méthode exec_() de QApplication (App étant le nom de l'instance). Il faut noter l'underscore dans le nom, pour éviter la confusion avec la fonction exec() de Python.

## V - Notes

En lançant cette application dans un environnement GNOME, on voit qu'un style GTK est utilisé. Qt peut s'adapter à l'environnment d'exécution et peut utiliser les styles natifs de la plateforme. Aussi, on peut utiliser l'option -style en ligne de commance pour le changer.

Ce programme n'est qu'un rapide début pour l'impatient, une dose pour l'hyperactif. On prendra une approche lente mais lisse pour les détails de PyQt au fur et à mesure que le tutoriel progresse.

## VI - Autre programme de test

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-

import sys
try:
 from PyQt4.QtCore import *     # Moteur contrôle Qt
 from PyQt4.QtGui import *     # IHM Qt
except:
 import time       # Gestion heures système
 for i in range(1, 11):
  print "PyQt non installé - A vérifier (%d/10)" % i
 time.sleep(5)
 sys.exit(1)
# try

class QtAppli(
  QApplication):     # Objet hérité
 "Fenêtre de l'application"

 # Constructeur fenêtre
 def __init__(
   self,      # Instance objet
   argv,      # Arguments programme
   transFile=None):    # Fichier de traduction

  # Message de contrôle
  print "Python version %s - QtAppli (qt v%s, pyqt_v%s)" % (
   sys.version,
   QT_VERSION_STR,
   PYQT_VERSION_STR
  )
```


Copyright ® 2009 Harsh. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts. Cette page est déposée à la SACD.
http://tcuvelier.developpez.com/tutoriels/pyqt/introduction-pyqt/

```
 # Appel constructeur de l'objet hértié
 QApplication.__init__(self, argv)

 # Translations
 if transFile != None:
  self.__trans=QTranslator()
  self.installTranslator(self.__trans)
  self.__trans.load(transFile)
 #if

 # Widget principale
 self.__mainWid=QMainWindow()
 self.__mainWid.setCentralWidget(QWidget(self.__mainWid))
 self.__mainWid.statusBar()

 # Titre
 self.__mainWid.setWindowTitle(
  self.trUtf8(
   "Vérification Qt (v%1)",
   "Note: titre de la fenêtre",
  ).arg(QT_VERSION_STR)
 )

 # Le bouton
 btn=QPushButton(
  self.trUtf8(
   "Surtout ne pas cliquer là !!!",
   "Note: titre du bouton",
  ),
  self.__mainWid.centralWidget()
 )
 self.connect(
  btn,
  SIGNAL("clicked()"),
  self.__slotAction,
 )

 # Le bouton version
 ver=QPushButton(
  self.trUtf8(
   "A propos de Qt",
   "Note: titre du bouton",
  )
 )
 self.connect(
  ver,
  SIGNAL("clicked()"),
  self.__slotQt,
 )

 # Pour quitter
 quit=QPushButton(
  self.trUtf8(
   "Quitter",
   "Note: titre du bouton",
  )
 )
 self.connect(
  quit,
  SIGNAL("clicked()"),
  self.__mainWid,
  SLOT("close()"),
 )

 # Rangement des éléments
 mainLayout=QVBoxLayout(self.__mainWid.centralWidget())
 mainLayout.addWidget(btn)
 mainLayout.addWidget(ver)
 mainLayout.addWidget(quit)
# __init__()

 # Affichage et lancement application
```


Copyright ® 2009 Harsh. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts. Cette page est déposée à la SACD.

```
 def run(
   self):     # Instance objet
  self.__mainWid.show()
  self.exec_()
 # run()

 # Slot qui affiche une fenêtre avec un texte
 def __slotAction(
   self):     # Instance objet

  print "%s.__slotAction" % self.__class__.__name__

  # Sous-fenêtre
  dial=QDialog(self.__mainWid.centralWidget())
  dial.setModal(True)
  dial.setWindowTitle(
   self.trUtf8(
    "Félicitations, Qt fonctionne parfaitement !!!",
    "Note: titre de la fenêtre",
   )
  )

  # Widget principale
  mainLayout=QVBoxLayout(dial)

  # Texte à la con
  lab=QLabel(
   self.trUtf8(

  "<center><font size='+5'>C'était écrit <u><font color='red'>SURTOUT</font></u> ne pas cliquer !!!</font><br>Et
    "Note: texte de la fenêtre",
   )
  )
  mainLayout.addWidget(lab)

  # Bouton
  btn=QPushButton(
   self.trUtf8(
    "Félicitations, Qt fonctionne parfaitement !!!",
    "Note: titre du bouton",
   )
  )
  btn.connect(
   btn,
   SIGNAL("clicked()"),
   dial,
   SLOT("close()"),
  )
  mainLayout.addWidget(btn)

  # Affichage sous-fenêtre
  dial.show()
 # __slotAction()

 # Slot qui affiche la version de Qt
 def __slotQt(
   self):     # Instance objet

  print "%s.__slotQt" % self.__class__.__name__

  # Fenêtre "A propos de Qt"
  QMessageBox.aboutQt(
   self.__mainWid,
   self.trUtf8(
    "à propos de Qt...",
    "Note: titre de la fenêtre",
   ),
  )
 # __slotQt()
# class QtAppli

if __name__ == "__main__":
```


Copyright ® 2009 Harsh. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts. Cette page est déposée à la SACD.

```
 # Lancement appli
 Appli=QtAppli(
  sys.argv,
  # "nom_du_fichier_de_traduction_éventuel",
 )
 Appli.run()
# if
```

## VII - Remerciements

Merci à Harsh pour l'autorisation de traduire son article, **The PyQt Intro** !

Merci à **Jean-Philippe André** pour sa relecture orthographique !

Merci à **Sve@r** pour son code de test, repris dans la section VI !


Copyright ® 2009 Harsh. Aucune reproduction, même partielle, ne peut être faite de ce site et de l'ensemble de son contenu : textes, documents, images, etc. sans l'autorisation expresse de l'auteur. Sinon vous encourez selon la loi jusqu'à trois ans de prison et jusqu'à 300 000 € de dommages et intérêts. Cette page est déposée à la SACD.
http://tcuvelier.developpez.com/tutoriels/pyqt/introduction-pyqt/